COMPUTER PROGRAMMING

❑ Learning Outcome 1:

Understand the differences between common programming languages

❑ DEFINITIONS

- **Computer program**
- **Computer programmer**
- **Computer programming**

❑ Common programming languages

- ❖Java
- ❖C
- ❖ C++
- ❖ Python
- ❖PHP
- ❖ Perl
- ❖ Ruby

❑ Computer program

- **A computer program** is a sequence of instructions written using a Computer Programming Language to perform a specified task by the computer.

OR

- A Computer program is a sequence of symbols that specifies a computation.
- **The two important terms that we have used in the above definition are:**
  1. Sequence of instructions
  2. Computer Programming Language

❑ Computer Programmer

- **Computer programmer** is Someone who can write computer programs or in other words, someone who can do computer programming.

❑ Computer programming(Language)

- ❖**A Computer programming language** is an artificial language designed to communicate instructions to a machine, particularly a computer.
- ❖**Programming languages** can be used to create programs that control the behaviour of a machine and/or to express algorithms.
- ❖**Any programming language** is composed of a set of predefined words that are combined according to predefined rules (syntax) to generate a computer program.

- The concepts of parsing and tokenisation, byte-code, assembly language, source code, machine code

❑ The concepts of parsing and tokenisation

❑ Parsing

- ❖ **Parsing** is a frequently used term both in the realm of data quality, and in computing in general.
- ❖ It can mean anything from simply 'breaking up data' to full Natural Language Parsing (NLP), which uses sophisticated artificial intelligence to allow computers to 'understand' human language.
- ❖ Parsing is designed to be used by developers of data quality processes to create packaged parsers for the understanding and transformation of specific types of data –
- ❖ For example Names data, Address data, or Product Descriptions.

❑ Tokenisation

- **Tokenization** is the act of breaking up a sequence of strings into pieces such as words, keywords, phrases, symbols and other elements called **tokens.**
- **Tokenization** is used in computer science, where it plays a large part in the process of lexical analysis.
- In the process of tokenization, some characters like punctuation marks are discarded. The tokens become the input for another process like parsing and text mining.
- Tokens themselves can also be separators. For example, in most programming languages, identifiers can be placed together with arithmetic operators without white spaces.
- **A token** is a piece of data that is recognized as a unit by the Parse processor using rules

❑ Byte-code

- **Bytecode** are a type of programming language that fall under the categories of both compiled and interpreted languages because they employ both compilation and interpretation to execute code.
- **Java** and the **.Net framework** are easily the most common examples of bytecode languages.
- In a bytecode language, the first step is to compile the current program from its human-readable language into bytecode.
- **Bytecode** is a form of instruction set that is designed to be efficiently executed by an interpreter and is composed of compact numeric codes, constants, and memory references.
- The largest benefit of bytecode languages is platform independence which is typically only available to interpreted languages,
- **Bytecode** can also make use of this technique to enhance execution speed

❑ Assembly language,

- **Assembly language** is a low-level programming language for a computer, or other programmable device, in which there is a very strong correspondence between the language and the architecture's machine code instructions.
- **Assembly language** is converted into executable machine code by a utility program referred to as an *assembler*
- **Assembly Language** introduced a mnemonic representation, using symbols such as ADD, MOV or POP to use rather than the binary codes.

❑ Source code

- **Source code** is any collection of computer instructions, possibly with comments, written using a *human-readable* programming language, usually as plain text (i.e., human readable alphanumeric characters).
- **The source code** of a program is specially designed to facilitate the work of computer programmers, who specify the actions to be performed by a computer mostly by writing source code.
- **The source code** is often transformed by an assembler or compiler into binary machine code understood by the computer.
- The machine code might then be stored for execution at a later time. Alternatively, source code may be interpreted and thus immediately executed.

❑ Machine code

- **Machine code** or **machine language** is a set of instructions executed directly by a computer's central processing unit (CPU).
- The CPU loads in one instruction at a time, and executes that instruction. The next instruction in sequence is then loaded in.
- Each instruction is represented as a fixed number of bits (in early days, 8 bits were used, although currently 64 bit instructions are common).
- The first 4 bits represented the operator (
  - e.g., 00 for NOOP, 01 for ADD, 02 for MOV, etc.
- The remaining 4 bits represent the data to operate on.
  - e.g. 01 01 Add 1 to the current sum.

- The role of the compiler in terms of input, output, and intermediate representations

- A compiler is a special program that processes statements written in a particular programming language and turns them into machine language or "code" that a computer's processor uses.
- Typically, a programmer writes language statements in a language such as Pascal or C one line at a time using an *editor*.
- Traditionally, the output of the compilation has been called *object code* or sometimes an *object module.*
- The object code is machine code that the processor can execute one instruction at a time.
- The programmer then runs the appropriate language compiler, specifying the name of the file that contains the source statements.
- When executing (running), the compiler first parses (or analyzes) from the input and then process the code to produce output for immediate presentations all of the language statements syntactically one after the other and then, in one or more successive stages or "passes", builds the output code, making sure that statements that refer to other statements are referred to correctly in the final code

❑ The role of an interpreter

❑**Definition**

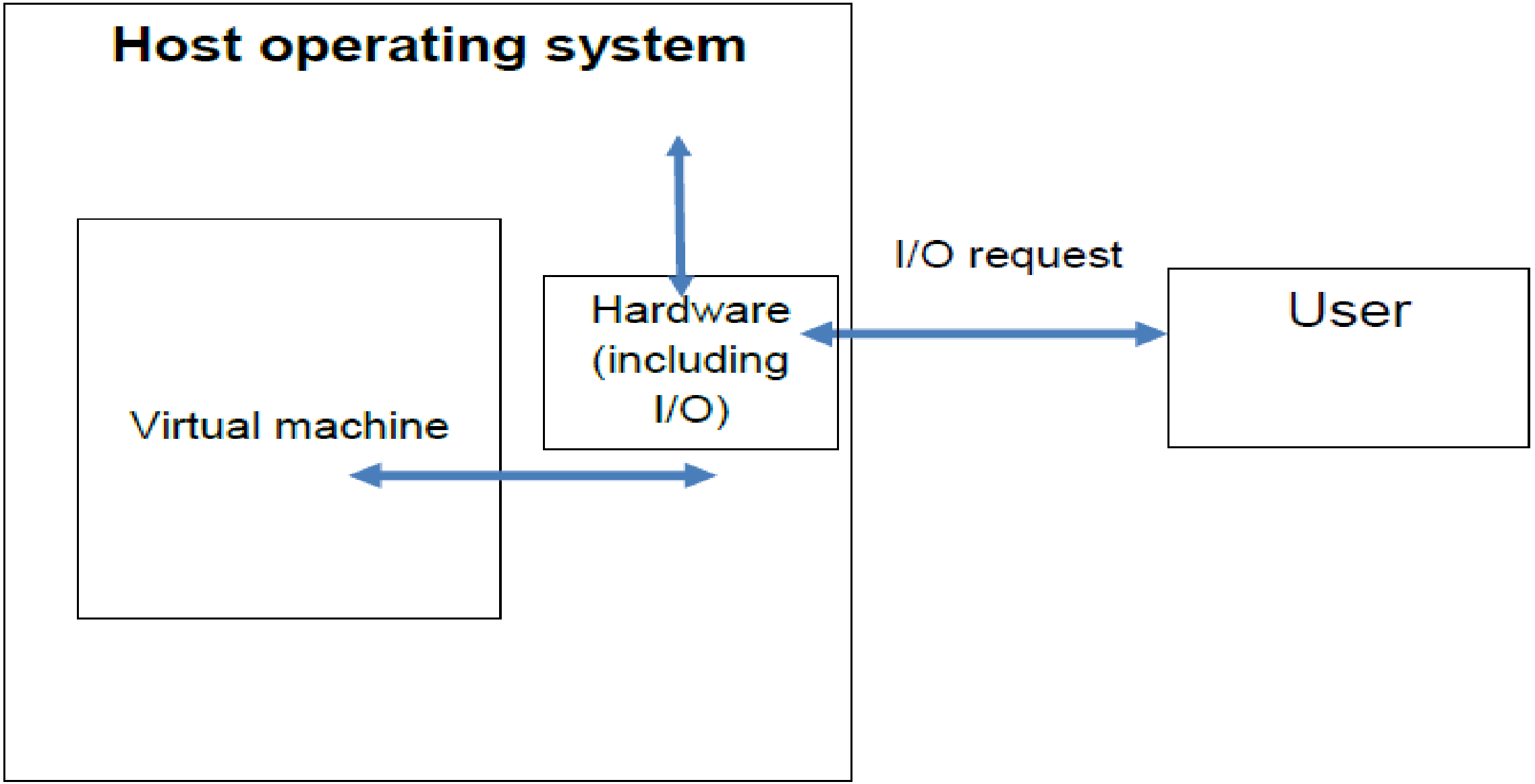
- ❖An interpreter is a computer program that is used to directly execute program instructions written using one of the many high-level programming languages.
- ❖Interpreter transforms or interprets a high-level programming code into code that can be understood by the machine (machine code).
- ❖The interpreter reads each statement of code and then converts or executes it directly

- An interpreter generally uses one of the following strategies for program execution:
  1. parse the source code and perform its behavior directly;
  2. translate source code into some efficient intermediate representation and immediately execute this;
  3. explicitly execute stored precompiled code[1] made by a compiler which is part of the interpreter system.

The role of a virtual machine (such as the Java Virtual Machine)

Definition

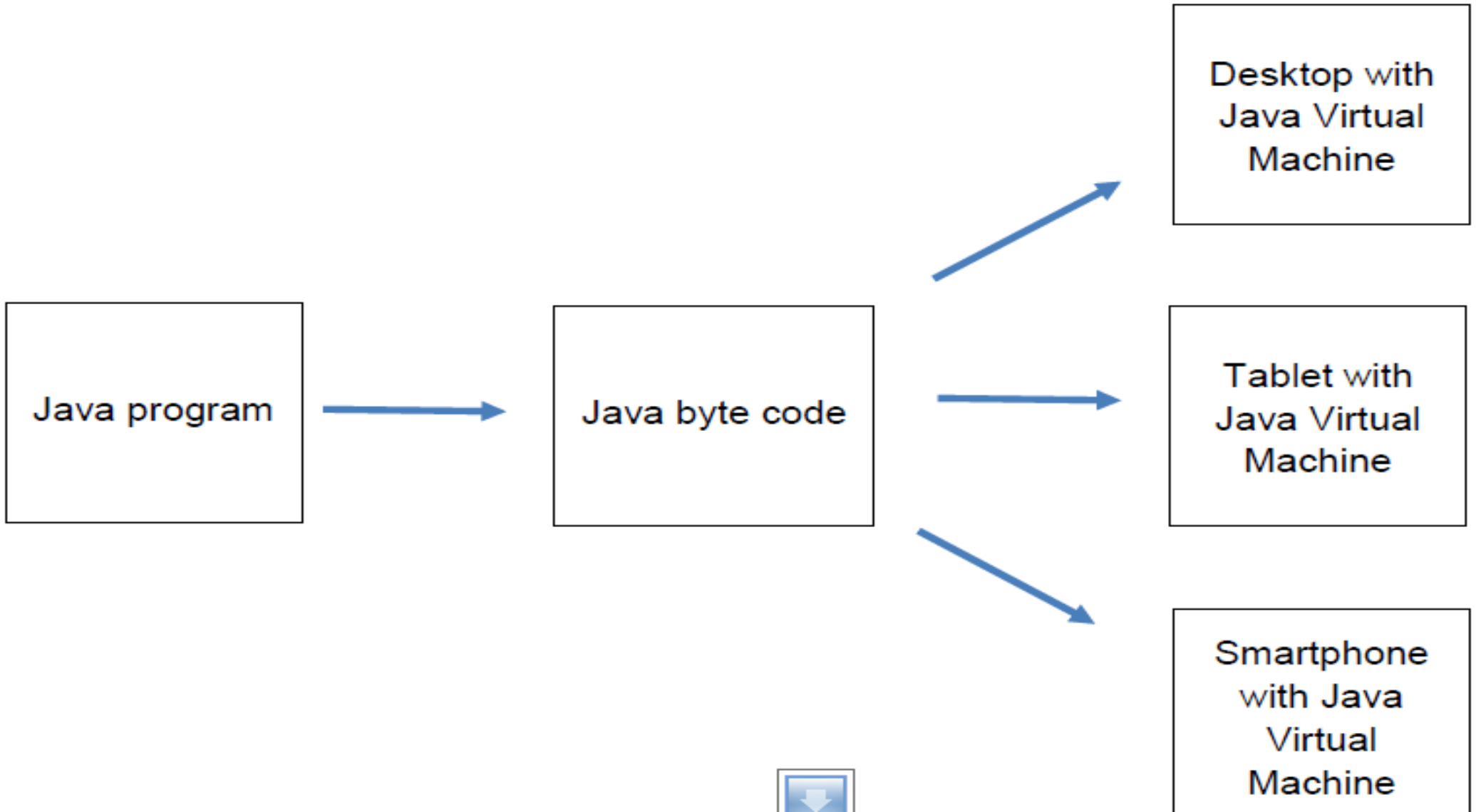
- **Virtual Machine is a** software to emulate the other computer system so that our computer can behave like that system.
- The process of creating and running a virtual machine is known as **virtualisation**.
- Virtualisation software, known as an **emulator**
- The emulator translates the instructions of the guest computer so that the host computer can understand them.
- The emulator also provides a bridge that allows the virtual machine to use the user interface, via the host operating system (I/O – input/output), allowing the user to interact with the software running on the virtual machine.


Host operating system
Virtual machine
Hardware (including I/O)
I/O request
User

❑ Example of Virtual Machine

- The Java Virtual Machine
  - **The Java Virtual Machine** is a virtual machine that reads and translates byte code into different machine codes for many different processor architectures and operating systems.
  - This means that **Java programs** can be run on any computer that has this virtual machine, including desktop PCs, laptops, tablets, smartphones and smart TVs.

❑Diagram show how Java virtual works



- Benefits and limitations of virtual machines

❑ Using virtual machines brings several benefits:

- ❖ They allow modern systems to use programs that run on obsolete or unsupported platforms
- ❖ They allow platform-independent programs to run on multiple platforms
- ❖ They allow one computer to behave as several computers simultaneously
- ❖ They can be easily backed up, copied and re-installed. This has particular benefits in corporate environments where network downtime can lead to loss of revenue. The virtual machines can be installed on another computer and be up and running again quickly.

❑ Virtual machines also have limitations:

- ❖ They emulate (copy) software, not hardware. Users trying to get the full functionality from software for another platform may find that they still cannot use the software correctly. For example, an emulation of a driving video game that requires a steering wheel might not be able to operate the game correctly
- ❖ Emulation requires processing power and memory. Running a virtual machine may take a substantial amount of these resources, resulting in reduced performance from the host computer
- ❖ If the host system lacks sufficient resources, the virtual machine will also suffer from poor performance and be difficult to use.

- Different methods of memory allocation in computer programming

Memory Allocation

- **Definition - What does *Memory Allocation* mean?**
- Memory allocation is a process by which computer programs and services are assigned with physical or virtual memory space.
- Memory allocation is the process of reserving a partial or complete portion of computer memory for the execution of programs and processes. Memory allocation is achieved through a process known as memory management.
- Memory allocation is primarily a computer hardware operation but is managed through operating system and software applications. Memory allocation process is quite similar in physical and virtual memory management.

- Programs and services are assigned with a specific memory as per their requirements when they are executed.
- Once the program has finished its operation or is idle, the memory is released and allocated to another program or merged within the primary memory.

- **Memory allocation has two core types;**

- Static Memory Allocation: The program is allocated memory at compile time.
- Dynamic Memory Allocation: The programs are allocated with memory at run time.

❑ The strengths and weaknesses of the different language implementation options

- ❖ Nowadays, C is mainly used in operating system programming and low level things, like drivers. It's super fast, but it also lacks some paradigms that make higher level languages easier to use.
- ❖ C++ is mainly used in games. It is about as low level as C, but with some added structures, notably classes and templates, which make programming a little bit easier for large teams. The language itself is harder to understand and reason about than C though.
- ❖ Javascript is the only language used in the browser to make client side web applications. Together with HTML and CSS, they form the languages of the Web. If you're going to make web programming, you must learn Javascript. Nowadays, you can also use Javascript on the server side.

- Ruby and Python are multipurpose scripting languages, mainly used on server side

2. Suitability of a language for a task based on its design and available language library

- The task for specific computation requirements (speed of computation, available memory, etc.)

- The amount and speed of the RAM in your computer makes a huge difference in how your computer performs.
- If you are trying to run Windows XP with 64 MB of RAM it probably won't even work.
- When the computer uses up all available RAM it has to start using the hard drive to cache data, which is much slower.
- The constant transfer of data between RAM and virtual memory (hard drive memory) slows a computer down considerably. Especially when trying to load applications or files.

- **The two types of RAM.**
- **Dynamic RAM** needs to be refreshed thousands of times per second. Dynamic RAM differ in the technology they use to hold data, dynamic RAM is being the more common type.
- **Static RAM** does not need to be refreshed, which makes it faster; but it is also more expensive than dynamic RAM.
- Both types of RAM are volatile, meaning that they lose their contents when the power is turned off.
- The normal speed of RAM in most computers today is pc100 (100mhz).

- This runs fine for most applications. Gamers or high-end machines probably are using DDR (double data rate) RAM.
- It's newer and more expensive, but runs considerably faster (266mhz). Note that all computers cannot use DDR RAM.

The likely environment the solution would execute in

- Environment it is the first step to be followed before setting on to write a program.
- When we say Environment Setup, it simply implies a base on top of which we can do our programming.
- Thus, we need to have the required software setup, i.e., installation on our PC which will be used to write computer programs, compile, and execute them. For example, if you need to browse Internet, then you need the following setup on your machine:
  - ✓ A working Internet connection to connect to the Internet
  - ✓ A Web browser such as Internet Explorer, Chrome, Safari, etc.

Key task constraints that would affect implementation choices (including reliability requirements, speed of delivery, flexibility, computation requirements)

ASSIGNMENT

The functional requirements and identifying library functionality that could be used to implement part or all of the feature

Functional Requirements

- The Functional Requirements Specification documents the operations and activities that a system must be able to perform.

Functional Requirements should include:

- Descriptions of data to be entered into the system
- Descriptions of operations performed by each screen
- Descriptions of work-flows performed by the system
- Descriptions of system reports or other outputs
- Who can enter the data into the system
- How the system meets applicable regulatory requirements

- The Functional Requirements Specification is designed to be read by a general audience.
- Readers should understand the system, but no particular technical knowledge should be required to understand the document.

The different implementation choices for a task based on task constraints and suitability of environment

ASSIGNMENTS

Learning Outcome 2: Understand the similarities between common programming languages

1. Explaining and comparing common data structures

The concept of a primitive data type (such as integers, booleans)

Primitive Data type

- A Primitive data type, in programming, is a classification that specifies which type of value a variable has and what type of mathematical, relational or logical operations can be applied to it without causing an error.
- A string, for example, is a data type that is used to classify text and an integer is a data type used to classify whole numbers.

Primitive data type are as follows

- **Integers**
- **Booleans**
- **Floating point type**
- **Character type**
- **void type**

Integers

- **Integers** are used to store whole numbers.
- Integer is a fundamental variable type built into the compiler and used to define numeric variables holding whole numbers.
- C, C++, C# and many other programming languages recognize int as a type. Other data types include float and double

Integer Types

The following table provides the details of standard integer types with their storage sizes and value ranges –

Type	Size(bytes)	Range
int or signed int	2	-32,768 to 32767
unsigned int	2	0 to 65535
short int or signed short int	1	-128 to 127
unsigned short int	1	0 to 255
long int or signed long int	4	-2,147,483,648 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295

- Given below is an example to get the size of int type on any machine.

```
#include <stdio.h>
#include <conio.h>

int main() {
   printf("Storage size for int : %d \n", sizeof(int));

   getch();
}
```

A program to find a leap year

```
#include<stdio.h>
#include<conio.h>
void main()
{
int year;
printf("Enter the year\n");
scanf("%d", & year);
if ((year%4)==0)
{
printf("Leap year");
}
else
{
printf("is not a year");
}
getch();
}
```

Booleans

- The Boolean type represents the values true and false.
- Although only two values are possible, they are rarely implemented as a single binary digit for efficiency reasons.
- Many programming languages do not have an explicit Boolean type, instead interpreting (for instance) 0 as false and other values as true.
- Boolean data simply refers to the logical structure of how the language is interpreted to the machine language.
- In this case a Boolean 0 refers to the logic False. True is always a non zero, especially a one which is known as Boolean 1.

Floating point type

Floating types are used to store real numbers.

Size and range of floating type on 16-bit machine

Type	Size(bytes)	Range
Float	4	3.4E-38 to 3.4E+38
double	8	1.7E-308 to 1.7E+308
long double	10	3.4E-4932 to 1.1E+4932

Character type

- **Character types** are used to store characters value.

Size and range of Character type on 16-bit machine

Type	Size(bytes)	Range
char or signed char	1	-128 to 127
unsigned char	1	0 to 255

Void type

- **Void type** means no value.
- This is usually used to specify the type of functions which returns nothing.
- We will get acquainted to this datatype as we start learning more advanced topics in C language, like functions, pointers etc.

The concept of a composite type (imaginary numbers, etc.)

- A **composite type** is any data type which can be constructed in a program using the programming language's primitive data types and other composite types.
- It is sometimes called a **structure** or **aggregate data type**, although the latter term may also refer to arrays, lists, etc.
- The act of constructing a composite type is known as composition.

- **An array** stores a number of elements of the same type in a specific order. They are accessed randomly using an integer to specify which element is required (although the elements may be of almost any type). Arrays may be fixed-length or expandable.
- **A list** is similar to an array, but its contents are strung together by a series of references to the next element.

The use and implementation of reference types such as pointers and function references

Pointers

- A **pointer** is a value that designates the address (i.e., the location in memory), of some value.

OR

- **Pointers** are variables that hold a memory location.
- Since the memory address are the locations in the computer memory where program instructions and data are stored, pointers be used to access and manipulate data stored in the memory.

Uses of pointers

- Pointers are used to return more than one value to the function.
- Pointers are more efficient in handling the data in data.
- Pointers reduce the length and complexity of the program
- They increase the execution speed.
- The pointers save data storage space in memory

There are four fundamental things you need to know about pointers:

- How to declare them (with the address operator “&” : int *pointer=&variable)
- How to assign to them (pointer = Null);
- How to reference the value to which the pointer points (known as *dereferencing*, by using the dereferencing operator “*”:value = *pointer:)
- How they relate to arrays (the vast majority of arrays in C are simple lists, also called "1 dimensional arrays", but we will briefly cover multi-dimensional arrays with some pointers in a later chapter).

Declaring pointers

- **Consider the following snippet of code which declares two pointers:**

```
struct MyStruct {
 int   m_aNumber;     float num2;
 };
int main()
{
int *pJ2;
 struct MyStruct *pAnItem;
}
```

- Lines 1-4 define a structure. Line 8 declares a variable which points to an int, and line 9 declares a variable which points to something with structure MyStruct.
- So to declare a variable as something which points to some type, rather than contains some type, the asterisk (*) is placed before the variable name.

A simple example to understand how to access the address of a variable without pointers?

In this program, we have a variable num of int type. The value of num is 10 and this value must be stored somewhere in the memory, right? A memory space is allocated for each variable that holds the value of that variable, this memory space has an address.

For example we live in a house and our house has an address, which helps other people to find our house. The same way the value of the variable is stored in a memory address, which helps the C program to find that value when it is needed.

So let's say the address assigned to variable num is 0x7fff5694dc58,which means whatever value we would be assigning to num should be stored at the location: 0x7fff5694dc58.

```c
#include <stdio.h>
#include <conio.h>
int main()
{    int num = 10;
printf("Value of variable num is: %d", num);
 /* To print the address of a variable we use %p
format specifier and ampersand (&) sign just
* before the variable name like &num.    */
printf("\nAddress of variable num is: %p", &num);
return 0;
}
```

Output

```
Value of variable num is: 10
Address of variable num is: 0x7fff5694dc58
```

Advantages of pointers

- Pointers are more compact and efficient code
- Pointers can be used to return multiple values from a function using its function arguments.
- They increase the execution speed and thus reduce the program execution time
- Pointers also provide an alternate way to access an array element.
- Pointers enable us to access the memory directly.

Function

- A function is a group of statements that together perform a task.
- Every C program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.

Declaring functions

- A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

```
return_type function_name( parameter list ) {
 body of the function}
```

Parts of a function

- **Here are all the parts of a function**
- **Return Type** − A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.
- **Function Name** − This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters** − A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.

- **Function Body** − The function body contains a collection of statements that define what the function does.

Example

```
/* function returning the max between two numbers */
int max(int num1, int num2) {
 /* local variable declaration */
 int result;
if (num1 > num2)
   result = num1;
else
result = num2;
return result; }
```

Function Declarations

- A function **declaration** tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately.
- A function declaration has the following parts –

```
return_type function_name( parameter list );
```

- For the above defined function max(), the function declaration is as follows −

```
int max(int num1, int num2);
```

Example of a function

```
#include<stdio.h>
void swap(int *a, int *b);
 int main()
{
   int m = 22, n = 44;
      printf("values before swap m = %d \n and n = %d",m,n);
   swap(&m, &n);
}
 void swap(int *a, int *b)
{
   int tmp;
   tmp = *a;
   *a = *b;
   *b = tmp;
   printf("\n values after swap a = %d \nand b = %d", *a, *b);
}
```

Output

values before swap m = 22

and n = 44
values after swap a = 44
and b = 22

The design and implementation of common collection structures: arrays, linked lists, dictionaries/maps, trees

Arrays

- **An array** is defined as **finite ordered collection of homogenous** data, stored in contiguous memory locations.
- **Here the words,**
  - **finite** *means* data range must be defined.
  - **ordered** *means* data must be stored in continuous memory addresses.
  - **homogenous** *means* data must be of similar data type.

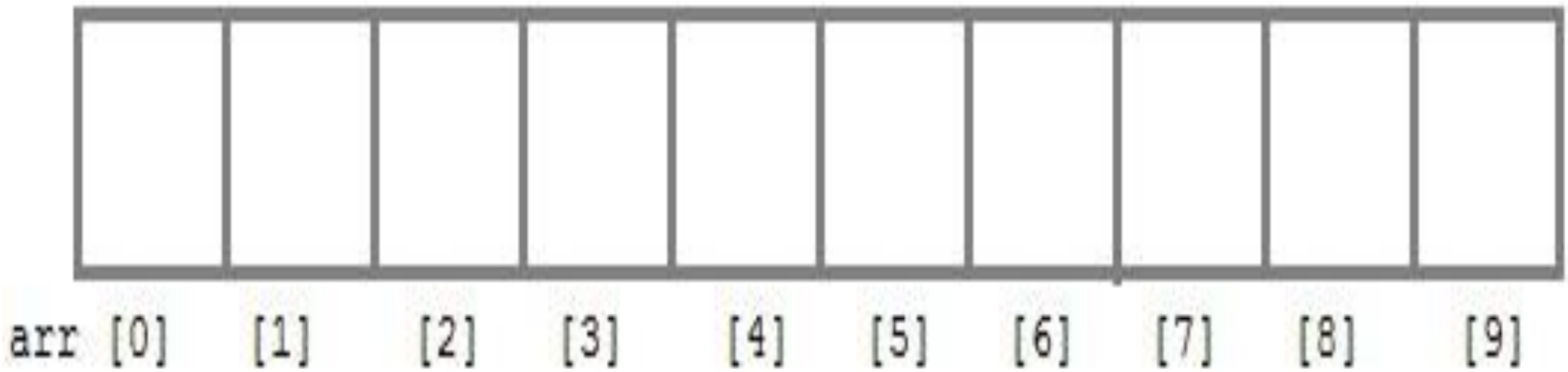
Example where arrays are used,

- to store list of Employee or Student names,
- to store marks of students,
- or to store list of numbers or characters etc.

Declaring an Array

- **Arrays** must be declared before they are used.
- General form of **array declaration** is,

```
data-type variable-name[size];
/* Example of array declaration */
int arr[10];
```


arr [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

- Here int is the data type, arr is the name of the array and 10 is the size of array.
- It means array arr can only contain 10 elements of int type.

Initialization of an Array

- After an array is declared it must be initialized. Otherwise, it will contain **garbage** value(any random value).
- An array can be initialized at either **compile time** or at **runtime**.

❑ The general form of initialization of array is,

```
data-type array-name[size] = { list of values };
/* Here are a few examples */
int marks[4]={ 67, 87, 56, 77 };    // integer
array initialization
float area[5]={ 23.4, 6.8, 5.5 };  // float
array initialization

int marks[4]={ 67, 87, 56, 77, 59 };
  // Compile time error
```

Example of array initialization

```c
#include<stdio.h>
#include<conio.h>
void main()
{
 int i;     int arr[] = {2, 3, 4};   // Compile time array initialization
 for(i = 0 ; i < 3 ; i++)
 {
 printf("%d\t",arr[i]);
 }
}
```

Output: 234

Types of Array

- **One dimensional Arrays**
- **Two dimensional Arrays**
- **Multi dimensional Arrays**

One dimensional Arrays

- It is otherwise called as vector.
- It is a list item contains or consists only one variable name.

Example

Int name [5]

Example program

```
#include <stdio.h>
 int main()
{
int i;
int a[5] ;
int *p = a;     // same as int*p = &a[0]
 for (i = 0; i < 5; i++)
 {
 printf("%d", *p);
        p++;
    }
    return 0;
}
```

Two dimensional Arrays

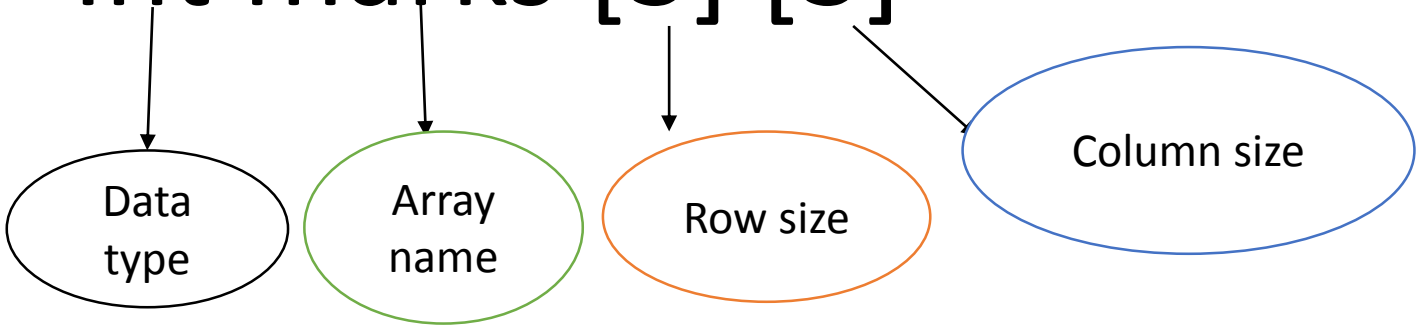
- It is otherwise called as Matrix
- It is a list item consists of two variables or subscripts one dimension is row and other is column.

Declaration

Type variable name [row size] [column size]

Example

Int marks [3] [3]



Multi dimensional Arrays

- It is a list item consist of three or more dimensions

Linked lists

A linked list is a linear data structure that consist of a sequence of data records.

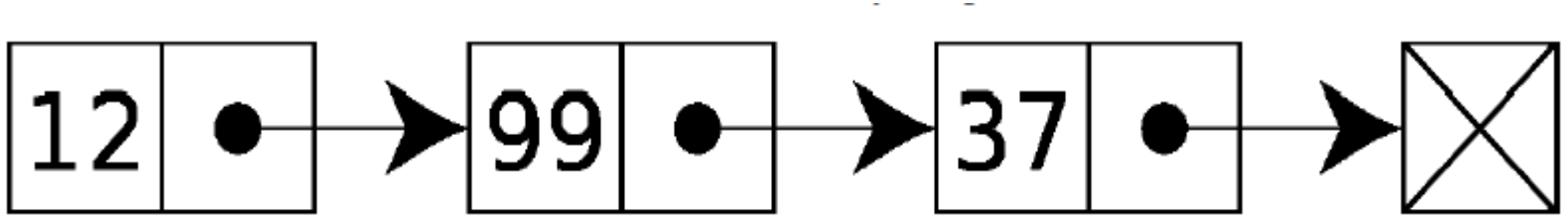
Such that each records there is a field that contains a reference to the next in the sequence.

Linked list can be singly, doubly or multiply linked and can either be linear or circular.

Basic Properties

- Objects, called **nodes**, are linked in a linear sequence
- A reference to the first node of the list is always kept. This is called the 'head' or 'front'

Example of link lists



A linked list with three nodes contain two fields each: an integer value and a link to the next node

Advantages of linked list

- Memory for a node is dynamically allocated, so the number of items that are added to a list is limited only by the amount of memory space.
- No wastage of memory available
- It provides flexibility and efficiency rearrange the items.

Conventions of Linked List

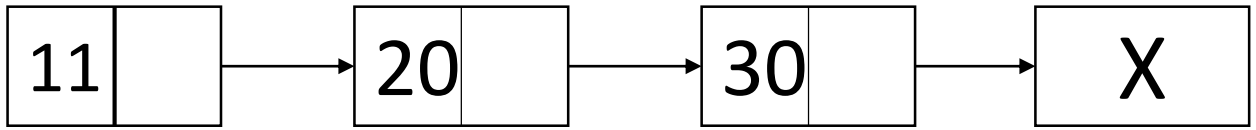
- There are several conventions for the linked list to indicate the end of list.
  - A null link
    - This means that points to no node (0) or null
  - A dammy node
    - This means that contains no item.
  - A reference back to the first node, making it a circular list.

Types of Linked List

- Singly linked list
- doubly Linked list
- multiply linked list

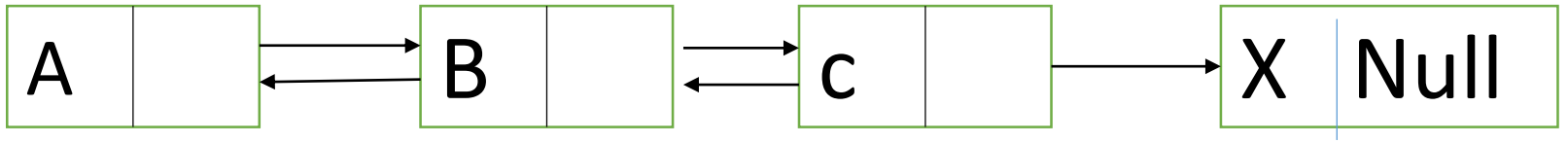
Singly linked list

- Is a linear data structure in which consist a pointer field that point to the address of its next node and the data items.



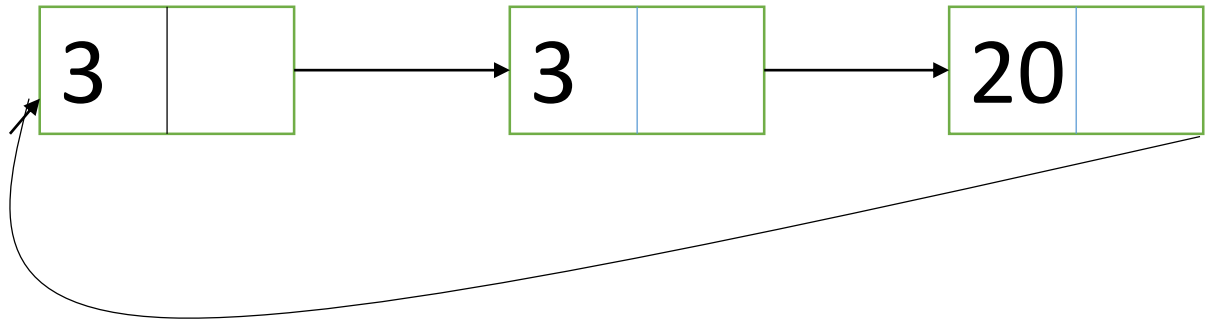
doubly Linked list

- Is a linear data structure that which consists two links or pointer field.



multiply linked list (circular linked list

- It is a list data structure in which last node point to first node



Dictionaries/Maps

- **Map** or a **Dictionary**) is an abstract data type composed of a collection of (key,value) pairs, such that each possible key appears at most once in the collection.

Operations associated with this data allow :

- ❖the addition of pairs (key, value) to the collection,
- ❖the removal of pairs from the collection,
- ❖ the modification of the values of existing pairs, and
- ❖the lookup of the value associated with a particular key

Map Operations

The operations that are usually defined for an associative array are;

- **Add** or **insert**: add a new (key,value) pair to the collection, binding the new key to its new value. The arguments

to this operation are the key and the value.

• **Reassign**: replace the value in one of the (key,value) pairs that are already in the collection, binding an old key to

a new value. As with an insertion, the arguments to this operation are the key and the value.

• **Remove** or **delete**: remove a (key,value) pair from the collection, unbinding a given key from its value. The

argument to this operation is the key.

• **Lookup**: find the value (if any) that is bound to a given key. The argument to this operation is the key, and the

value is returned from the operation. If no value is found, some associative array implementations raise an

exception.

Map Implementation

- For dictionaries/map with very small numbers of bindings, it may make sense to implement the dictionary using an association list, a linked list of bindings.
- With this implementation, the time to perform the basic dictionary operations is linear in the total number of bindings; however, it is easy to implement and the constant factors in its running time are small.
- Another very simple implementation technique, usable when the keys are restricted to a narrow range of integers, is direct addressing into an array:
- The most frequently used general purpose implementation of an associative array is with a hash table: an array of bindings, together with a hash function that maps each possible key into an array index.

Tree

- A **tree** is a widely-used data structure that emulates a hierarchical tree structure with a set of linked nodes.

Terminology of Tree

Node.

- **A node** is a structure which may contain a value, a condition, or represent a separate data structure (which could be a tree of its own).
- Each node in a tree has zero or more **child nodes.**
- A node that has a child is called the child's **parent node** (or *ancestor node*, or superior).
- A node has at most one parent.

An internal node or inner node

- An **internal node** or **inner node** is any node of a tree that has child nodes.
- Similarly, an **external node** (also known as an **outer node**, **leaf node**, or **terminal node**), is any node that does not have child nodes.
- The topmost node in a tree is called the **root node**. Being the topmost node, the root node will not have a parent.
- All other nodes can be reached from it by following **edges** or **links**.
- Every node in a tree can be seen as the root node of the subtree rooted at that node. A **free tree** is a tree that is not rooted.

The height of a node

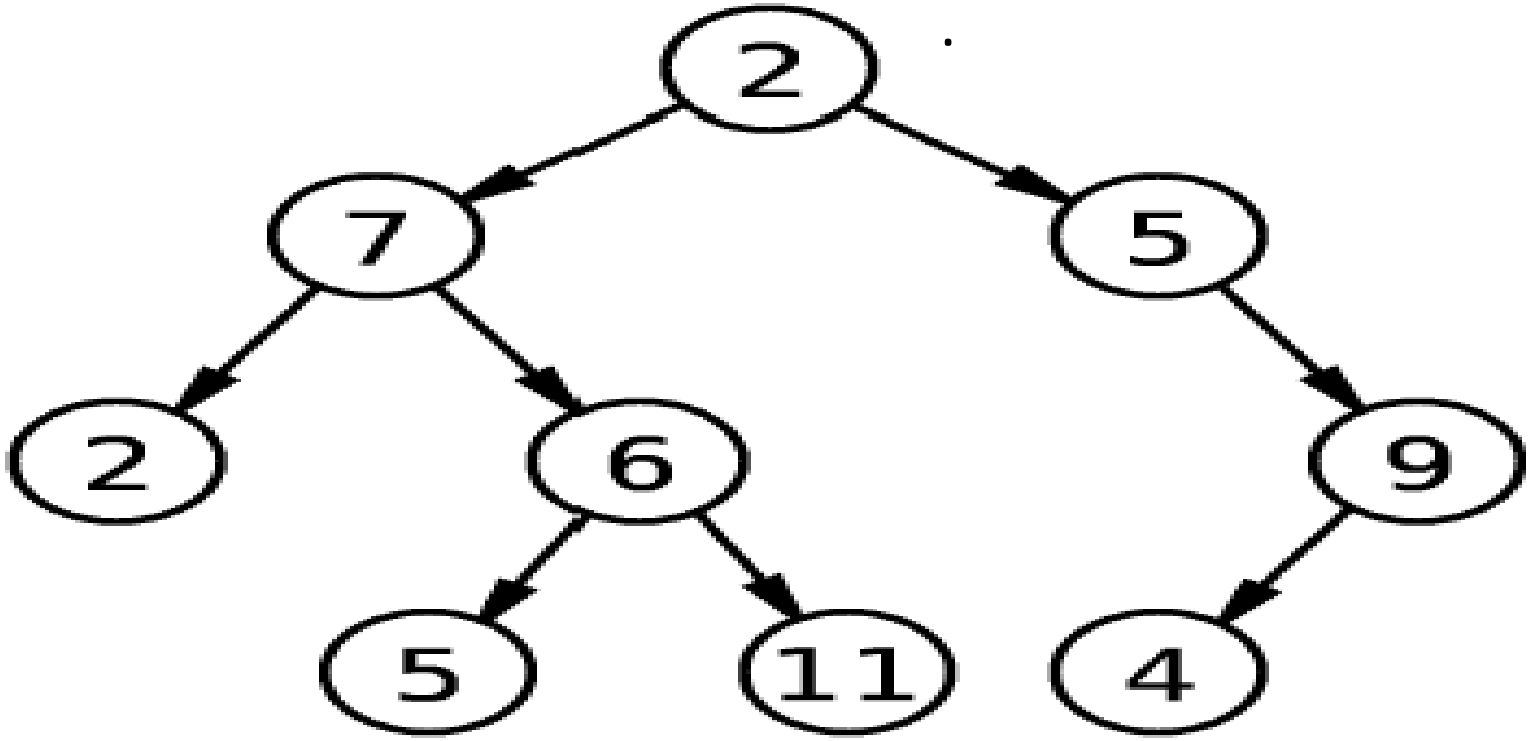
The **height** of a node is the length of the longest downward path to a leaf from that node.

The height of the root is the height of the tree.

The **depth** of a node is the length of the path to its root (i.e., its *root path*). This is commonly

A subtree of a tree *T* is a tree consisting of a node in *T* and all of its descendants in *T*. (This is different from the formal definition of subtree used in graph theory.

Example of Tree data structure



A simple unordered tree; in this diagram, the node labeled 7 has two children, labeled 2 and 6, and one parent, labeled 2. The root node, at the top, has no parent.

Representations

There are many different ways to represent trees; common representations represent the nodes as dynamically allocated records with pointers to their children, their parents, or both, or as items in an array, with relationships between them determined by their positions in the array (e.g., binary heap).

The process of adding, removing and searching for data items within common collection structures